cla method를 사용해서 그래프를 한번 지워주는 것이 좋습니다
https://cosmosproject.tistory.com/431

그런데 그래프를 그리는 method인 bar를 사용했으므로
위 animation은 사실 과거에 그려진 그래프가 겹쳐져있는 형태라고 보면 됩니다 (겹쳐져있기 때문에 결과물에서 보이지는 않지만
이렇게 되면 animation이 진행되는 단계가 길어질수록 겹쳐져있는 그래프가 많아질거고 따라서 animation 후반부에는 굉장히 진행 속도가 느려집니다.
그래서 이렇게 animation을 사용할 때 지속적으로 새로운 그래프를 그리는 형태로 animation을 구성할거라면
update된 새로운 그래프를 그리기 전에 cla method를 사용해서 그래프를 한번 지워주는 것이 좋습니다.

```
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
fig = plt.figure()
list_x = [1, 2, 3, 4, 5]
list_y = [0, 0, 0, 0, 0]
line = plt.bar(list_x, list_y)
plt.xlim(0, 6)
plt.ylim(0, 20)

def update(x_coordinate):
    list_y[x_coordinate] = x_coordinate + 1
    plt.cla()
    plt.bar(list_x, list_y)
graph_ani = FuncAnimation(fig=fig, func=update, frames=[0, 1, 2, 3, 4], interval=300, repeat=True)
plt.show()
```

참고로 cla method는 title, x label, y label 등 좌표평면 자체를 제외한 모든것을 삭제하므로
title, xlim, ylim 등의 설정이 원하는대로 유지되길 원한다면 cla method가 실행된 이후에 한번 더 설정되도록 해야합니다.

(cla 관련 내용 = https://cosmosproject.tistory.com/438)

1) Python matplotlib : animation FuncAnimation (matplotlib animation, 그래프 애니메이션으로 그리기, graph animation)

CosmosProject 2022. 1. 16. 01:28

matplotlib는 단순히 결과 그래프를 보여주는 것 뿐 아니라 그래프가 그려지는 과정을 animation으로 나타내주는 기능을 제공합니다.

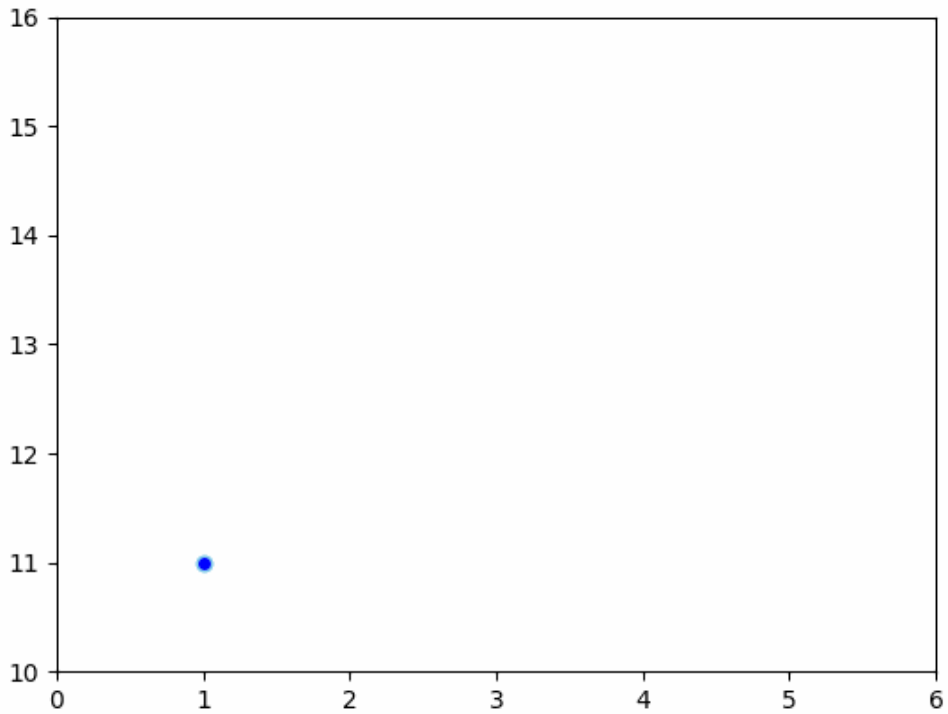
이 기능을 사용하기 위한 기본적인 사항들을 이해하기 위해선 코드 단위별로 살펴봐야할 것이 있으므로,

먼저 전체 코드를 본 후에 각 부분이 어떤 것을 의미하고 왜 그렇게 쓰였는지 이해해봅시다.

```
import matplotlib.pyplot as plt

from matplotlib.animation import FuncAnimation
```

```
fig = plt.figure()


line = plt.plot([], [],
                color='skyblue',
                marker='o', markerfacecolor='blue',
                markersize=6)[0]
plt.xlim(0, 6)
plt.ylim(10, 16)


list_x = []
list_y = []
def update(x_coordinate):
    list_x.append(x_coordinate)
    list_y.append(x_coordinate + 10)


    line.set_data(list_x, list_y)
```

```
graph_ani = FuncAnimation(fig=fig, func=update, frames=[1, 2, 3, 4, 5])

graph_ani.save('graph_ani.gif', writer='imagemagick', fps=3, dpi=100)


plt.show()
```



위 코드를 실행하면 gif움짤처럼 그래프가 그러지는 과정이 실시간으로 업데이트되며 마치 동영상처럼 보입니다.

matplotlib에서 그래프를 그리는 과정을 animation으로 나타내는 방식은 그냥 좌표점을 하나씩 찍고 이어주는 과정을 보여주는겁니다.

이제 위 코드의 각 부분이 어떤 것을 의미하는지 체크해봅시다.

먼저 matplotlib에서 animation을 나타내기 위해선 **FuncAnimation** method를 사용하는데,

FuncAnimation method에서 필요한 3개의 인자가 있습니다.

그 3가지는 다음과 같습니다.

1. fig = 그래프의 설정(plt.figure) 관련 정보

2. func = 좌표를 1개씩 update할 때 마다 그래프를 update하기 위해 사용할 함수

3. frame = 2번의 func 함수가 실행될 때 마다 2번의 func 함수의 argument로서 전달할 값의 list

즉, 우리는 **FuncAnimation** method를 사용하기 전에 위 3개의 요소를 미리 다 정의해놔야한다는 것입니다.

이제 위 코드를 한줄씩 보죠.

fig = plt.figure()

Step 1.

FuncAnimation method에 사용될 첫 번째 요소를 얻기 위한 부분입니다.

plt.figure()는 그래프의 크기를 설정해주는 등 그래프의 외관을 설정해주는 기능을 가집니다.

(figure의 figsize option을 활용하는 내용 = https://cosmosproject.tistory.com/425?category=1001293)

보통 일반적인 그래프를 그릴 때에는 plt.figure()만 코드에 적어주면 그래프에 설정이 잘 적용되지만 이번에는 figure 정보를 FuncAnimation에 전달해야하므로 figure method로부터 return되는 figure 객체를 fig라는 변수에 할당하였습니다.

지금 볼 예시에서는 animation을 그리는데 집중하기 위해 figure method에 아무 값도 넣지 않고 그래프의 설정(figure)이 모두 기본값으로 되게 하였습니다.

하지만 animation에서도 일반 그래프를 그릴 때 처럼 figure에 여러 가지 설정을 할 수 있습니다.

https://cosmosproject.tistory.com/425

figure에 figsize를 설정해주면 output되는 animation 그래프도 설정한 figsize로 생성됩니다.

```
line = plt.plot([], [],
                color='skyblue',
                marker='o', markerfacecolor='blue',
                markersize=6)[0]
```

Step 2.

두 번째로는 비어있는 그래프를 그립니다.

plt.plot의 x축, y축 값을 비어있는 list의 형태 []로 전달하였습니다.

이것의 의미는 말 그대로 그냥 아무 좌표가 없는 그래프인 그냥 비어있는 좌표평면을 그리겠다는 것이죠.

평소에 그래프를 그리기만 한다면 plt.plot만 사용하면 됩니다.

그러나 우리는 animation을 사용하면서 plt.plot으로 그린 그래프의 좌표값을 계속해서 update할 것이므로 현재 존재하는 그래프에 대한 정보가 필요합니다.

따라서 비어있는 좌표평면에 대한 정보를 얻기 위해 plt.plot method로 return되는 그래프 정보를 line이라는 변수에 할당하였습니다.

(plt.plot method는 기본적으로 그려진 그래프의 정보를 return한다고만 알아둡시다.)

근데 여기서 한가지 주의할 점이 있습니다.

plt.plot의 가장 마지막 부분을 보면 markersize=6)[0]

이 부분처럼 맨 마지막에 [0] 이라는 부분이 보이시나요?

```
 line = plt.plot([], [],
                 color='skyblue',
                 marker='o', markerfacecolor='blue',
                 markersize=6)
```

뭔가 이상하죠.

plt.plot method로 인해 return되는 무언가를 line이라는 변수에 할당하려면 그냥 위처럼 적으면 되는데 plt.plot 맨 마지막에 [0]을 적어서 마치 list에 indexing을 하여 index = 0인 값을 line이라는 변수에 할당하는 그림입니다.

결론부터 말하자면 indexing하기 위한 것이 맞습니다.

이 부분을 좀 더 살펴보죠.

```
import matplotlib.pyplot as plt
```

```
line = plt.plot([], [],
                color='skyblue',
                marker='o', markerfacecolor='blue',
                markersize=6)


print(line)
print(type(line))
print(len(line))
print(line[0])
```

-- Result

```
[<matplotlib.lines.Line2D object at 0x7f82a889fd60>]
<class 'list'>
1
```

Line2D(_child0)

자 위의 코드를 별도로 작성하여 실행해보았습니다.

보시면 plt.plot으로 비어있는 좌표평면을 그리는 것은 동일하죠.

그리고 plt.plot method의 결과로서 return되는 어떤 값을 line이라는 변수에 할당했고, 과연 plt.plot method는 어떤 데이터를 return하는지 print해보았습니다.

print(line) --> 결과값 = [<matplotlib.lines.Line2D object at 0x7fcf39aded60>]

line에 할당된 데이터는 **plt.plot** method가 return하는 그 자체의 데이터입니다.

결과를 보니 **[<matplotlib.lines.Line2D object at 0x7fcf39aded60>]** 라는 값이 나옵니다.

정확한 것은 모르겠지만 matplotlib의 line 2D 객체(object)를 return했네요.

print(type(line)) --> 결과값 = <class 'list'>

그리고 이 부분이 중요합니다.

위 print의 결과가 **<class 'list'>** 로 나왔습니다.

이 말은 **plt.plot** method가 matplotlib의 line 2D 객체를 return했는데 이 객체를

list에 담아서 return했다는 의미인 것이죠.

print(len(line)) --> 결과값 = 1

그러면 이 list의 길이가 몇인지 보니까 1이라고 합니다.

print(line[0]) --> 결과값 = Line2D(_child0)

line 변수에 할당된 matplotlib line 2D 객체의 길이가 1이므로 index = 0인 값을 추출해보았더니 **Line2D(_child0)** 라는 값이 나옵니다.

즉, 우리가 진짜 원하는 **plt.plot** method가 return하는 그래프에 대한 정보는 바로 **line[0]**에 있는 값인 것이죠.

```
line = plt.plot([], [],
                color='skyblue',
                marker='o', markerfacecolor='blue',
                markersize=6)[0]
```

자 그러면 다시 이 부분을 봅시다.

plt.plot method를 이용해서 비어있는 좌표평면을 그렸고,

우리는 animation 기능을 통해 이 비어있는 좌표평면에 좌표를 하나씩 update해 줄 것이므로,

plt.plot method에서 return되는 그래프에 관한 정보가 필요한데

이 정보는 **plt.plot** method에서 return되는 list의 형태로 된 line 2D 객체의 index = 0에 있는 값이다 라는 의미입니다.

```
plt.xlim(0, 6)

plt.ylim(10, 16)
```

Step 3.

그 다음 부분은 x축의 최대/최소 범위와 y축의 최대/최소 범위를 적어주는겁니다.

이걸 적어주지 않으면 정상적으로 animation을 볼 수 없습니다.

일단 그 이유를 간단히 설명하자면,

우리는 보통 일반적으로 그래프를 그릴 때 여러 좌표들의 값을 전달합니다.

그러면 matplotlib는 주어진 점들의 x값들과 y값들을 감지하여 x축의 최대/최소값, y축의 최대/최소값을 어느정도 적절하게 자동으로 정해줍니다.

근데 우리는 plt로 아무 좌표가 없는 비어있는 그래프를 먼저 그렸고, 이때 matplotlib는 아무 좌표(점)가 없기 때문에 이 그래프의 x축/y축의 최대/최소값을 자동으로 -0.04 ~ 0.04 정도로 설정해버립니다.

따라서 animation을 나타낼 때 x축 값이 1, 2, 3, 4, 5인 좌표값을 나타내려하는데 결과로 보여지는 그래프의 x축 범위는 -0.04 ~ 0.04이므로 그래프가 정상적으로 보일 리가 없죠.

따라서 xlim, ylim method를 사용해서 결과 그래프에 보여줄 x축의 최대/최소값과 y축의 최대/최소값의 범위를 지정해줘야 합니다.

Step 4.

설명 초반에 animation을 사용하기 위해서 필요한 요소는 다음 3가지라고 하였습니다.

1. fig = 그래프의 설정(plt.figure) 관련 정보

2. func = 좌표를 1개씩 update할 때 마다 그래프를 update하기 위해 사용할 함수

3. frame = 좌표를 1개씩 update할 때 마다 사용될 x값 list

1번 fig는 Step 1에서 설정하였습니다.

이제 2번 func에 필요한 내용을 구성하는 부분입니다.

```
 list_x = []

list_y = []

def update(x_coordinate):

    list_x.append(x_coordinate)

    list_y.append(x_coordinate + 10)
```

```
line.set_data(list_x, list_y)
```

func에는 좌표를 1개씩 update할 때 사용할 로직을 의미합니다.

위 내용을 보면 update라는 이름의 함수를 설정하고있습니다.

update라는 함수는 x_coordinate(x좌표를 의미하는 변수명이라는걸 상징시키기 위해 그냥 제가 알기쉽게 지은겁니다.)라는 값을 받아서 list_x에 추가하고있고,

x_coordinate 값에 10을 더한 값을 y에 추가하고 있습니다.

이렇게 한 이유는 그냥 제가 y = x + 10 그래프를 그리고싶었기 때문이예요.

이제 y값을 담을 list(list_y)에 여러가지 함수나 로직을 적용하면 원하는 그래프를 다 그릴 수 있습니다.

그리고 이렇게 append된 list를 기준으로 line.set_data를 이용해서 우리가 그려놓은 비어있는 좌표평면에 새로운 좌표 정보를 넣어주는겁니다.

(set_data method 관련 내용 = https://cosmosproject.tistory.com/432)

이 함수가 어떻게 작동할지 미리 살펴보면

FuncAnimation method가 받는 인자 중에는 frames라는 이름의 인자가 있고, 이것은 일련의 값들이 담긴 list를 인자로 받습니다.

[1, 2, 3, 4, 5]라는 list를 받았다고 하면 이 list 속 값을 하나하나씩 참조합니다.

마치 for loop (for i in [1, 2, 3, 4, 5]) 처럼요.

그리고 이렇게 하나씩 참조한 값을 위에서 정의한 update function의 argument(x_coordinate)로 전달하여 update 함수를 실행합니다.

그러면 update 함수는 전달받은 frames list의 값들을 가지고 일련의 과정을 거쳐 x좌표 list, y좌표 list를 만든 후에 이 x값/y값 좌표를 set_data method에 전달하여 그래프의 좌표를 업데이트하게됩니다.

그러면 마치 그래프가 점점 그려지는 것처럼 보이는 것이죠.

(여기서 한 가지 오해가 발생할 수 있는데 frames에 전달된 list의 값이 반드시 x값이 될 필요는 없습니다. frames 인자를 통해 전달된 list는 단순히 update function을 실행시킬 때 update function의 argument로 들어가는 것일 뿐 이 사이에서 x축 값과 y축 값을 정하는 로직은 마음대로 구성할 수 있어요.)

여기서 한 가지 더 주의할 점은 x값과 y값을 담을 list인 list_x, list_y를 update 함수 밖에 선언한 것입니다.

만약 list_x, list_y를 update함수 안쪽에 선언한다면 FuncAnimation이 update함수를 실행할 때 마다 list_x, list_y가 비어있는 list로 초기화된 상태에서 새로운 x, y 좌표를 할당하므로 과거의 x값이 누적되지 않은 채로 새로운 값으로 update될겁니다.

이렇게 하면 점이 찍히면서 좌표들이 누적되며 그래프가 그려지는 것이 아니라, 점이 하나 찍히고 없어진 후 다음 점이 찍히는 형태의 animation이 생성됩니다.

```
graph_ani = FuncAnimation(fig=fig, func=update, frames=[1, 2, 3, 4, 5])
```

Step 5.

자 이제 FuncAnimation을 사용하기 위해 필요한 필수적인 요소들이 다 모였습니다.

fig인자에 전달하는 fig는 Step 1에서 정의하였습니다.

func인자에 전달하는 update는 Step 4에서 정의하였습니다.

한 가지 frames라는 인자가 있는데 이 frame는 animation에서 그려질 그래프의 x

좌표를 의미합니다.

위 예시에서는 [1, 2, 3, 4, 5]라는 list를 전달하였는데, 이 말은 animation으로 x 축 값이 1, 2, 3, 4, 5인 좌표 5개를 순차적으로 찍겠다는 의미입니다.

```
graph_ani.save('graph_ani.gif', writer='imagemagick', fps=3, dpi=100)
```

Step 6.

생성된 animation을 gif 파일(통칭 움짤)로 저장하기 위한 부분입니다.

'graph_ani.gif'는 생성할 gif 파일의 이름입니다.

writer 옵션은 imagemagick을 쓰면 됩니다.

fps는 frames per second인데 이 숫자가 높을수록 그래프가 그려지는 속도가 빠릅니다.

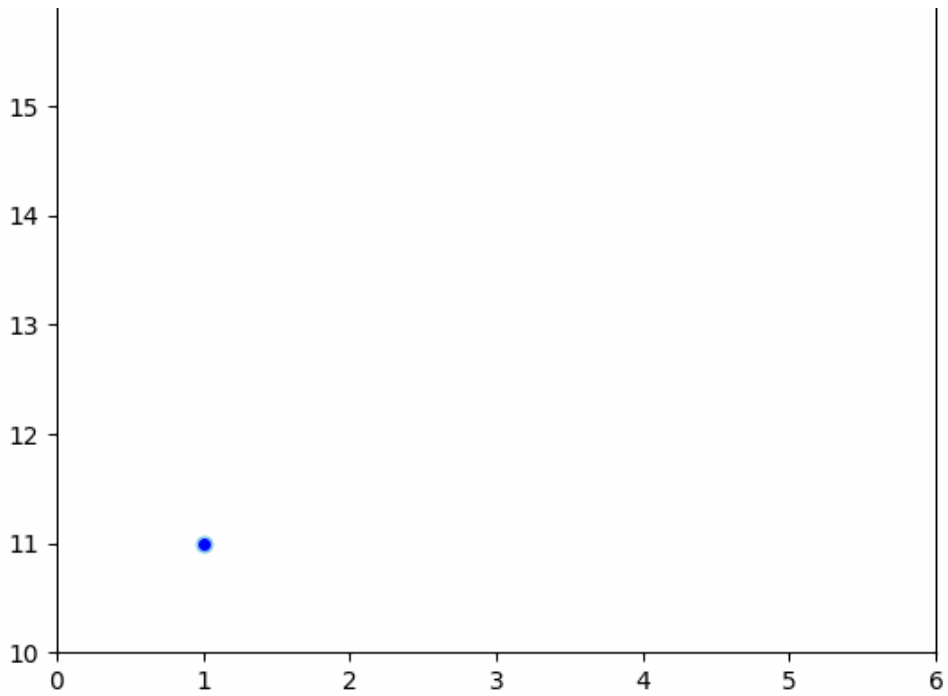
dpi(dots per inch)는 생성될 gif 파일의 해상도를 의미합니다.

만약 움짤 gif 파일을 생성하고 싶지 않다면 이 부분은 지워도 됩니다.

```
plt.show()
```

Step 7.

그리고 생성된 gif파일을 열거나 위처럼 plt.show() method를 이용해서 생성된 그래프를 열어보면 아래 이미지처럼 그래프가 움직이면서 그려지는 내용을 얻을 수 있습니다.



이제 기본적으로 matplotlib에서 animation을 사용할 수 있게 되었습니다.

추가로 필수적이진 않지만 FuncAnimation method에서 사용할 수 있는 옵션 몇가지를 봐봅시다.

```
 import matplotlib.pyplot as plt

from matplotlib.animation import FuncAnimation



fig = plt.figure()



line = plt.plot([], [],

                color='skyblue',

                marker='o', markerfacecolor='blue',

                markersize=6)[0]

plt.xlim(0, 6)

plt.ylim(10, 16)
```

```
def first_init():
    line.set_data([], [])


list_x = []
list_y = []
def update(x_coordinate):
    list_x.append(x_coordinate)
    list_y.append(x_coordinate + 10)

    line.set_data(list_x, list_y)

graph_ani = FuncAnimation(fig=fig, init_func=first_init, func=update, frames=[1, 2, 3, 4, 5])
graph_ani.save('graph_ani.gif', writer='imagemagick', fps=3, dpi=100)
```

```
plt.show()
```

먼저 **init_func**이라는 옵션입니다.

위에서 봤던 예시와 다른 점이 2가지가 있습니다.

```
def first_init():
    line.set_data([], [])

graph_ani = FuncAnimation(fig=fig, init_func=first_init, func=update, frames=[1, 2, 3, 4, 5])
```

다른 부분만 나타내면 위와 같습니다.

first_init이라는 함수를 생성하였고, FuncAnimation method의 init_func 인자에 first_init 함수를 전달하고 있습니다.

FuncAnimation method의 init_func 옵션은 animation을 그릴 때 가장 먼저 딱 한

번 실행되는 함수를 설정할 수 있습니다.

보통 animation 시작 전에는 좌표평면에 아무런 좌표가 없어야하겠죠?

그래서 set_index method에 x축/y축 값을 비어있는 list로 전달하여 그래프에 어떠한 좌표도 있지 않도록 하는 기능을 하는 first_init 함수를 만들고,

이렇게 만들어진 first_init함수를 FuncAnimation의 init_func 인자로서 전달하여 animation이 시작할 때 가장 처음 first_init 함수가 실행되게 하여 혹시나 있을 좌표를 모두 지워버리는 역할을 하는것이죠.

근데 사실 이게 큰 의미가 있는지는 의문입니다.

가장 처음 plot으로 비어있는 그래프를 그리기 때문에 굳이 init_func 옵션을 이같은 방식으로 사용할 효용성은 없어보입니다.

괜히 코드만 길어지는 것 같네요.

두 번째는 **interval** option입니다.

FuncAnimation method에서 사용할 수 있는 interval 옵션은 그래프를 그릴 때 하나의 좌표에서 다음 좌표를 찍을 때 까지의 시간 간격을 의미합니다.

즉, interval 옵션의 값이 크면 클수록 다음 좌표를 찍을 때 까지의 시간이 길어져서 그래프가 더 느리게 그려집니다.

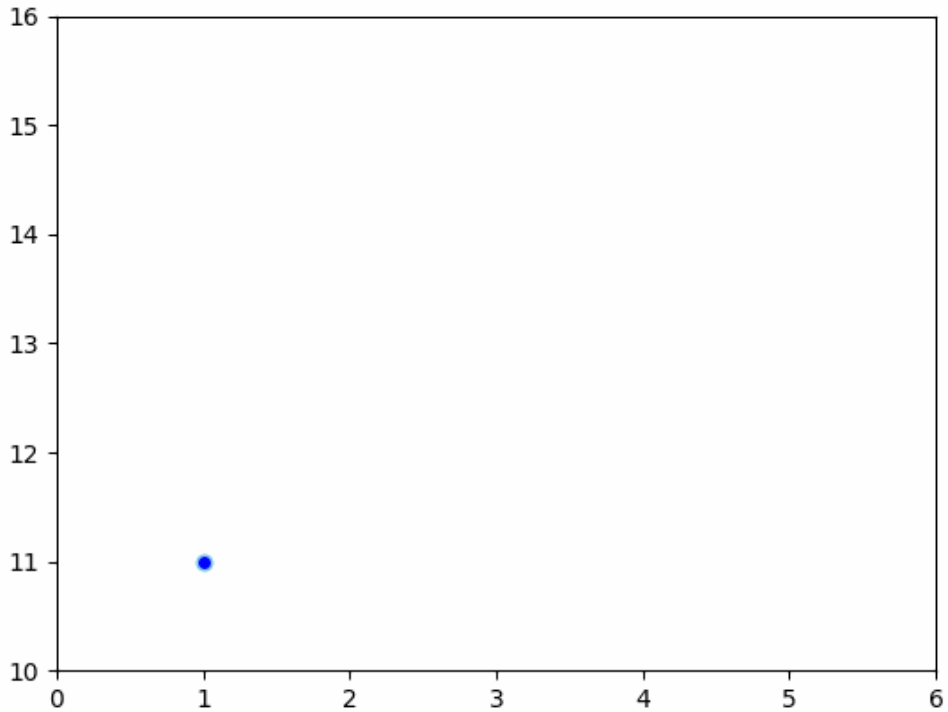
반대로 interval 옵션의 값이 작으면 작을수록 다음 좌표를 찍을 때 까지의 시간이 짧아져서 그래프가 더 빠르게 그려집니다.

```
 import matplotlib.pyplot as plt

from matplotlib.animation import FuncAnimation



fig = plt.figure()



line = plt.plot([], [],
```

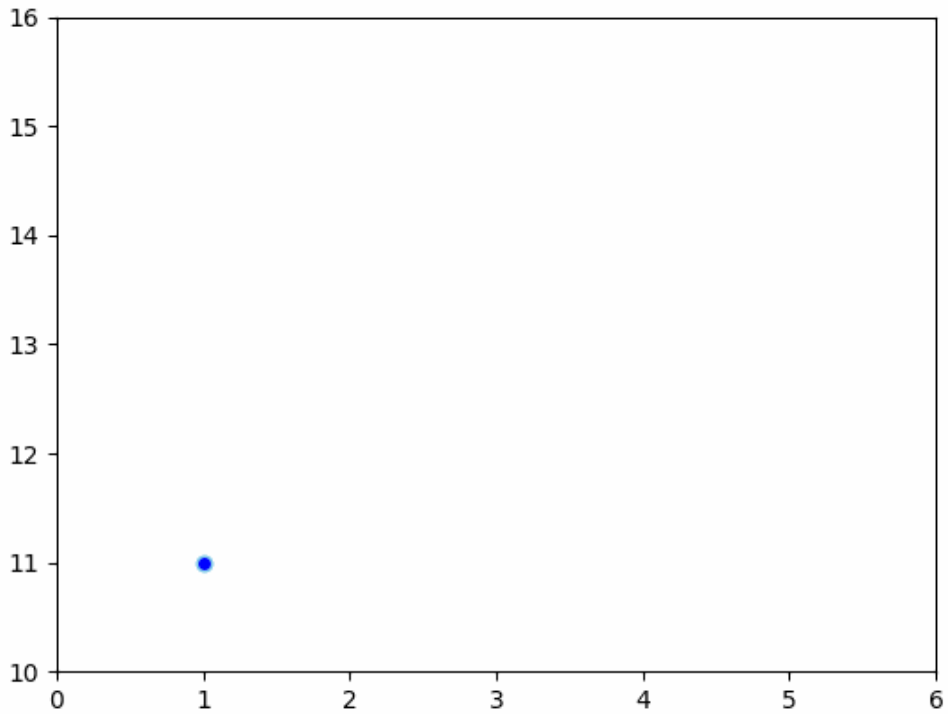
```
                    color='skyblue',
                    marker='o',  markerfacecolor='blue',
                    markersize=6)[0]
plt.xlim(0,  6)
plt.ylim(10,  16)


list_x  =  []
list_y  =  []
def  update(x_coordinate):
    list_x.append(x_coordinate)
    list_y.append(x_coordinate  +  10)


    line.set_data(list_x,  list_y)


graph_ani  =  FuncAnimation(fig=fig,  func=update,  frames=[1,  2,  3,  4,  5],
interval=100)
```

```
graph_ani.save('graph_ani.gif',  writer='imagemagick',  dpi=100)
```

```
plt.show()
```




```
inverval = 10
```



interval = 1000

interval을 10과 1000으로 설정했을 때 각각의 결과를 나타내는 이미지를 비교해 보면 그래프가 그려지는 속도의 차이가 확연히 느껴질겁니다.

여기서 한 가지 주의할 점은 이렇게 inverval로 속도를 조절해놨는데 save method를 이용해 gif 파일로 저장할 때 save method의 옵션 중 fps 옵션을 설정해버리면 inverval로 조절한 속도가 거의 무시되고 fps옵션이 거의 우선적으로 덮

어씌워집니다.

즉, interval 옵션을 이용해서 그래프가 그려지는 속도를 조절함과 동시에 이 속도를 그대로 반영시켜서 gif 움짤 파일도 생성하고싶으면 save method에 fps 옵션을 명시하지 말고 생략해야 합니다.

반대로 interval 옵션은 이용하지 않고 생성될 gif 움짤 파일에서만 그래프가 그려지는 속도를 조절하려면 save method에서 fps method를 이용하면 됩니다.

결국 FuncAnimation의 interval 옵션과 save method의 fps 옵션 둘 중 하나만 사용하라는 의미입니다.

근데 사실 save method의 fps 옵션은 생성될 gif 움짤 파일에만 속도 조절이 가능하므로 그냥 fps 옵션은 사용하지 말고 FuncAnimation의 interval 옵션만 사용하는 것을 추천합니다.

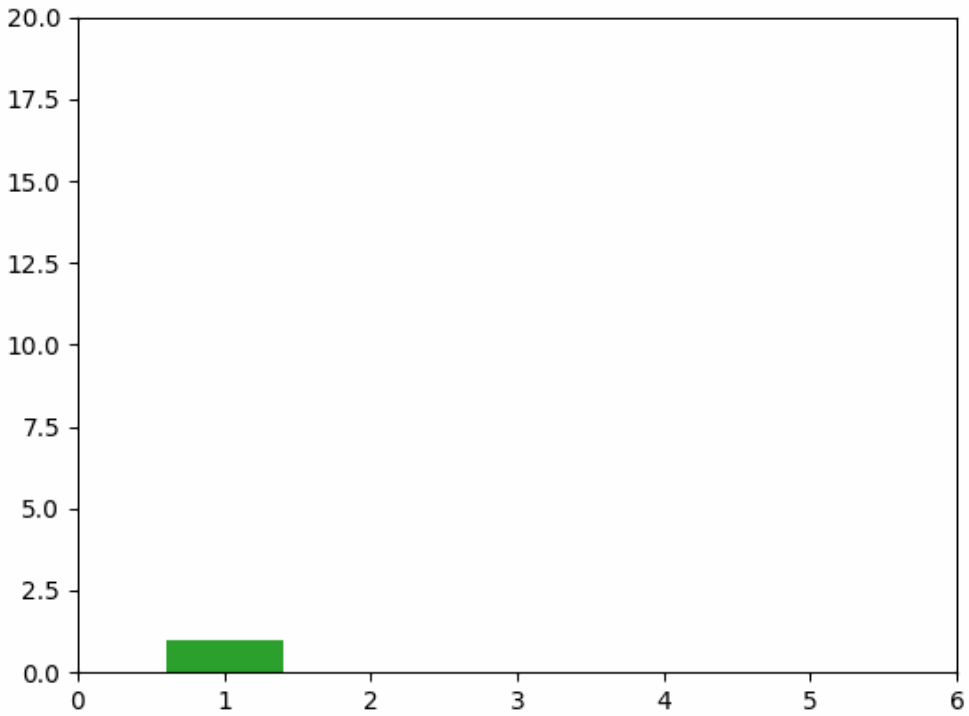
```
import matplotlib.pyplot as plt

from matplotlib.animation import FuncAnimation


fig = plt.figure()
```

```
list_x = [1, 2, 3, 4, 5]

list_y = [0, 0, 0, 0, 0]

line = plt.bar(list_x, list_y)



plt.xlim(0, 6)

plt.ylim(0, 20)



def update(x_coordinate):

    list_y[x_coordinate] = x_coordinate + 1



    plt.bar(list_x, list_y)



graph_ani = FuncAnimation(fig=fig, func=update, frames=[0, 1, 2, 3, 4],
interval=300, repeat=True)
```

plt.show()

세 번째는 repeat 옵션입니다.

repeat=True인 경우 frames에 제시된 값들을 계속 반복적으로 제시하여 그래프를 그리는 작업을 반복합니다.



위 코드는 bar graph를 그리는 과정을 FuncAnimation을 이용하여 애니메이션화 한 결과입니다.

frames에 [1, 2, 3, 4, 5]라는 5개의 값이 들어가있으니

예상하기로는 update 함수에 1, 2, 3, 4, 5 각각의 값을 전달하여 총 5번 실행하

게 될 것이고 이 값을 모두 전달하면 값의 전달을 멈출거라고 생각됩니다.

하지만 그렇지 않습니다.

repeat=True로 설정되어있기 때문에 frames에 제시된 [1, 2, 3, 4, 5]의 값을 반복적으로 전달하여 무한히 그래프를 그리며 animation을 나타내줍니다.

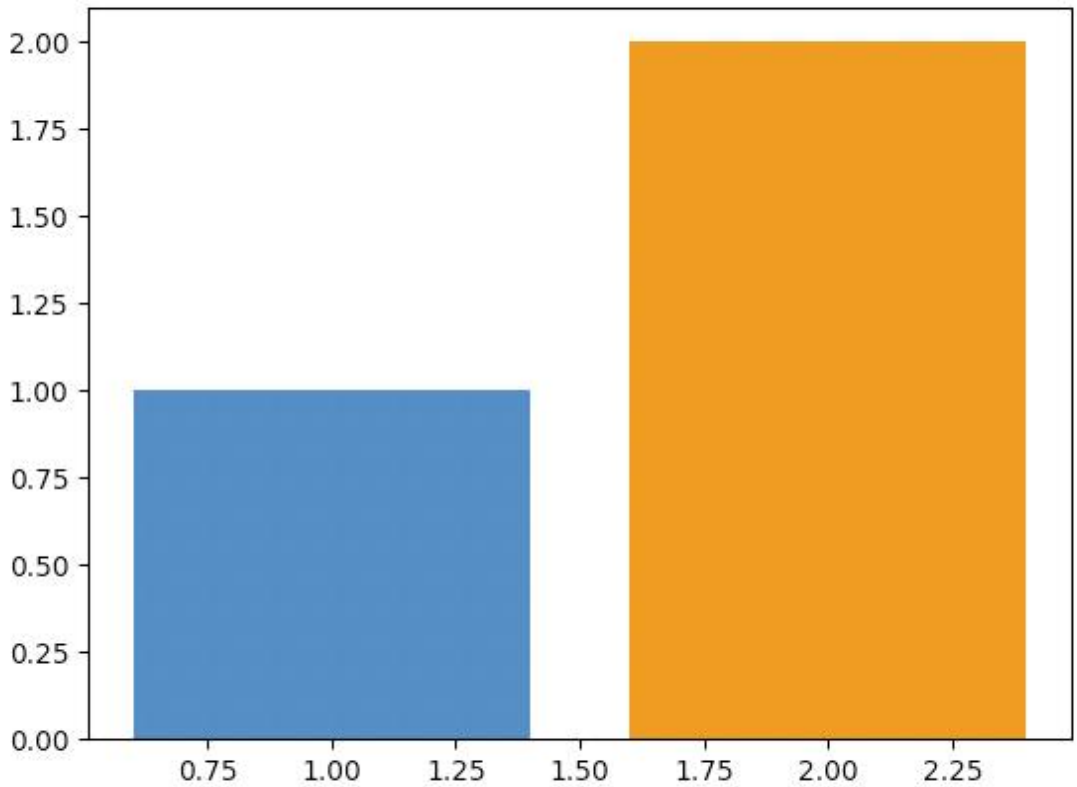
따라서 위 코드를 실행해보면 막대 5개가 그려진 후 계속해서 막대의 색상이 변할 겁니다.

즉, frames에 전달된 [1, 2, 3, 4, 5]의 값을 update 함수에 지속적으로 전달하며 계속해서 그래프를 그리고 있는 것입니다.

만약 위 코드에서 **repeat=False**로 설정을 바꿔주면 막대 5개가 그려진 후 딱 멈추는 것을 볼 수 있습니다.

그리고 animation을 나타낼 때 주의할 사항 한가지를 짚고 넘어가봅시다.

```
 import matplotlib.pyplot as plt



plt.bar([1], [1])

plt.bar([2], [2])



plt.show()
```



위 코드는 x = 1, y = 1인 막대 그래프를 먼저 그리고

그 다음에 x = 2, y = 2인 막대 그래프를 나중에 그렸습니다.

그런데 결과는 2개의 막대 그래프가 동시에 나타나게 됩니다.

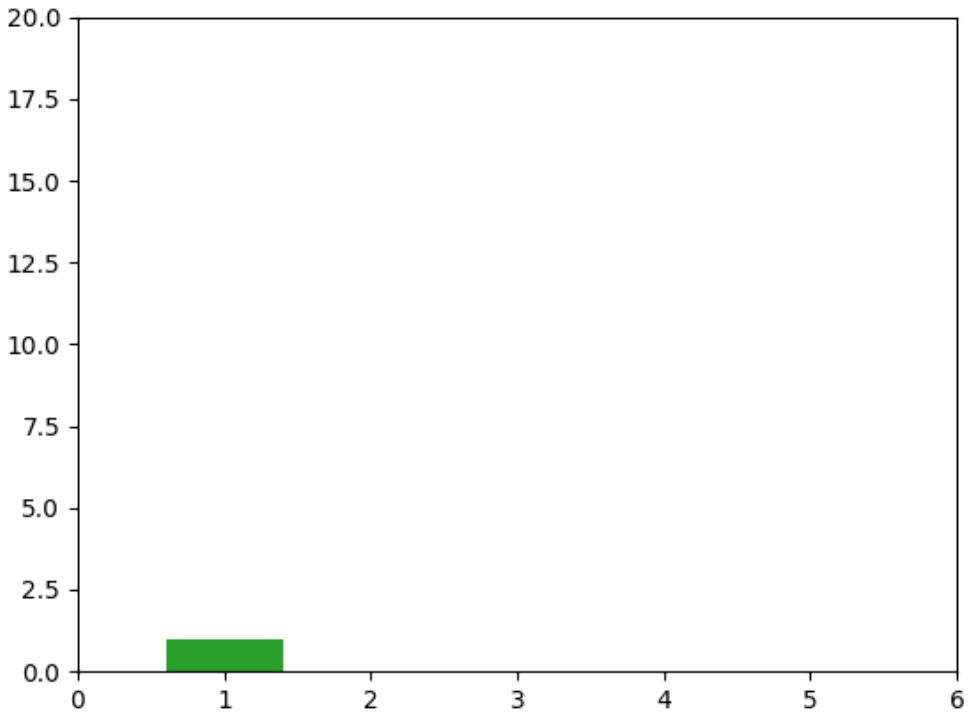
이렇게 matplotlib에서 그래프를 그리는 method(plot, bar, pie 등)를 통해 그려진 그래프는 사라지지 않고 남아있습니다.

따라서 위 예시처럼 이전에 그린 그래프의 정보가 그대로 남아서 나중에 그린 그래프와 동시에 나타나게 되는거죠.

animation method를 사용할 때에도 비슷한 상황이 벌어질 수 있습니다.

```
 import matplotlib.pyplot as plt

from matplotlib.animation import FuncAnimation



fig = plt.figure()



list_x = [1, 2, 3, 4, 5]

list_y = [0, 0, 0, 0, 0]

line = plt.bar(list_x, list_y)



plt.xlim(0, 6)

plt.ylim(0, 20)
```

```
def update(x_coordinate):

    list_y[x_coordinate] = x_coordinate + 1



    plt.bar(list_x, list_y)



graph_ani = FuncAnimation(fig=fig, func=update, frames=[0, 1, 2, 3, 4],
interval=300, repeat=True)



plt.show()
```



위에서 본 예시 중 하나인 위 animation도 보이는건 마치 막대가 하나씩 늘어나는 것 같지만 사실은 그게 아닙니다.

위 예시의 코드를 보시면 update 함수에서 **list_y[x_coordinate] = x_coordinate + 1** 이 부분을 통해 list_y를 업데이트하고,

그 다음 **plt.bar(list_x, list_y)**를 통해 update된 list_y를 가지고 다시 그래프를 그리는 것입니다.

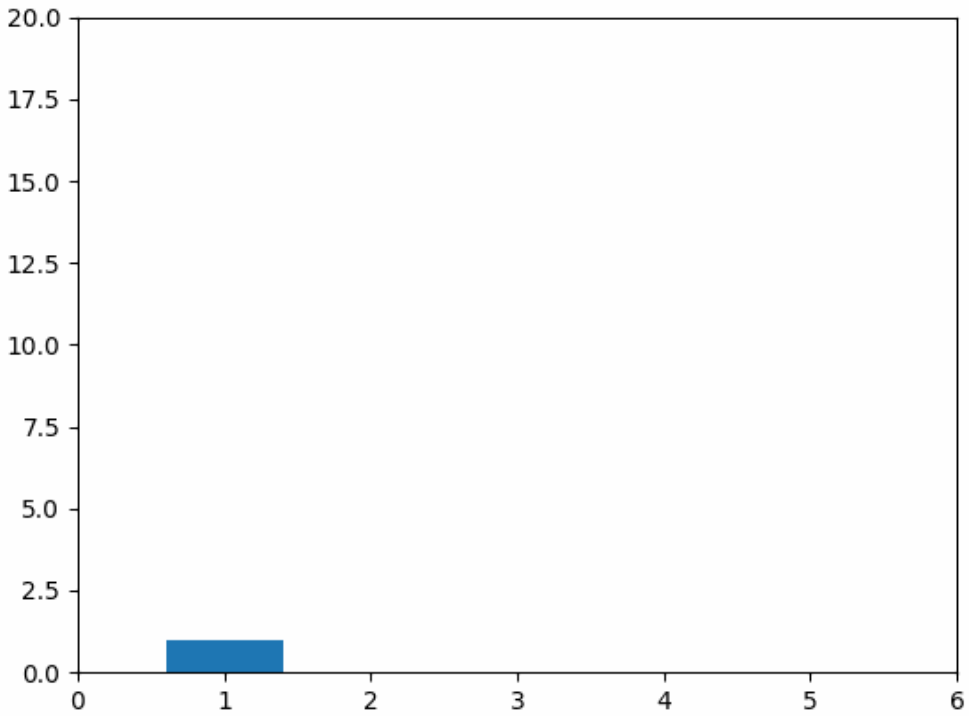
그런데 그래프를 그리는 method인 bar를 사용했으므로

위 animation은 사실 과거에 그려진 그래프가 겹쳐져있는 형태라고 보면 됩니다. (겹쳐져있기 때문에 결과물에서 보이지는 않지만요.)

이렇게 되면 animation이 진행되는 단계가 길어질수록 겹쳐져있는 그래프가 많아질거고 따라서 animation 후반부에는 굉장히 진행 속도가 느려집니다.

그래서 이렇게 animation을 사용할 때 지속적으로 새로운 그래프를 그리는 형태로 animation을 구성할거라면

update된 새로운 그래프를 그리기 전에 cla method를 사용해서 그래프를 한번 지워주는 것이 좋습니다.




```
import matplotlib.pyplot as plt

from matplotlib.animation import FuncAnimation
```

```
fig = plt.figure()


list_x = [1, 2, 3, 4, 5]

list_y = [0, 0, 0, 0, 0]

line = plt.bar(list_x, list_y)


plt.xlim(0, 6)

plt.ylim(0, 20)


def update(x_coordinate):

    list_y[x_coordinate] = x_coordinate + 1


    plt.cla()

    plt.bar(list_x, list_y)
```

```
graph_ani = FuncAnimation(fig=fig, func=update, frames=[0, 1, 2, 3, 4], interval=300, repeat=True)
```

```
plt.show()
```

참고로 cla method는 title, x label, y label 등 좌표평면 자체를 제외한 모든것을 삭제하므로

title, xlim, ylim 등의 설정이 원하는대로 유지되길 원한다면 cla method가 실행된 이후에 한번 더 설정되도록 해야합니다.

(cla 관련 내용 = https://cosmosproject.tistory.com/438)

여담이지만 아래 코드는 Galton board를 막대그래프로 구현하고 그것이 실시간으로 변하는 animation을 구현한 코드입니다.

(bar graph와 Galton board관련 예시 참고 링크 = https://cosmosproject.tistory.com/427)

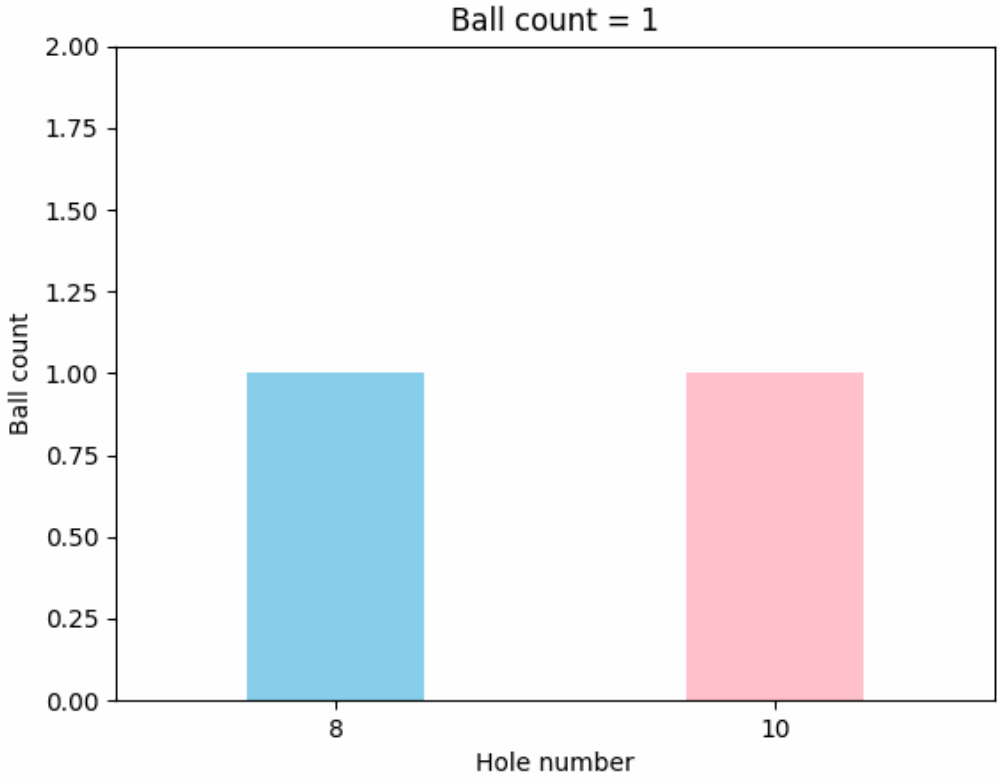
sample_cnt를 늘리면 늘릴수록 그래프는 정규분포에 가까워집니다.

```
import matplotlib.pyplot as plt

from matplotlib.animation import FuncAnimation

import numpy as np


sample_cnt = 500

result_max = 17

stair_cnt = result_max - 1


dict_result = {}


fig = plt.figure()
```

```
line = plt.bar([], [],
               color=['pink', 'skyblue', 'lightgreen', 'grey'])


def falling_down(x):
    val_result = (1 + result_max) / 2

    for s in range(stair_cnt):
        val_prob = np.random.rand()

        if val_prob >= 0.5:
            val_add_factor = 0.5
        elif val_prob < 0.5:
            val_add_factor = -0.5
        else:
```

```
            raise Exception('Prob error')


        val_result = val_result + val_add_factor


dict_result[val_result] = dict_result.get(val_result, 0) + 1


plt.clf()

plt.bar(list(dict_result.keys()), list(dict_result.values()),

            color=['pink', 'skyblue', 'lightgreen', 'grey'])

plt.xlabel('Hole number')

plt.ylabel('Ball count')

plt.title('Normal distribution')


val_x_min = min(list(dict_result.keys()))

val_x_max = max(list(dict_result.keys()))

val_y_min = min(list(dict_result.values()))
```

```
    val_y_max = max(list(dict_result.values()))


    plt.xlim(val_x_min - 1, val_x_max + 1)

    plt.ylim(val_y_min - 1, val_y_max + 1)

    plt.xticks(list(dict_result.keys()))


    plt.title('Ball count = {}'.format(x + 1))

    print('Ball count =', x + 1)


graph_ani = FuncAnimation(fig=fig, func=falling_down, frames=range(sample_cnt), interval=1, repeat=False)

plt.show()
```


Ball count = 1
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
Ball count
8
10
Hole number